

Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №3
Численное интегрирование
Вариант 5

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Зинченко Антон Андреевич,
Преподаватель практики.

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ	5
1. ВЫЧИСЛИТЕЛЬНАЯ РЕАЛИЗАЦИЯ ЗАДАЧИ	6
2. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ	8
ВЫВОД	12

Задание

$$\int_2^4 (-2x^3 - 3x^2 + x + 5)dx$$

Обязательное задание

Исходные данные:

1. Пользователь выбирает функцию, интеграл которой требуется вычислить (3–5 функций) из тех, которые предлагает программа.
2. Пределы интегрирования задаются пользователем.
3. Точность вычисления задается пользователем.
4. Начальное значение числа разбиения интервала интегрирования: $n = 4$.
5. Ввод исходных данных осуществляется с клавиатуры.

Программная реализация задачи:

1. Реализовать в программе методы по выбору пользователя:
 - Метод прямоугольников (3 модификации: левые, правые, средние)
 - Метод трапеций
 - Метод Симпсона
2. Методы должны быть оформлены в виде отдельной(ого) функции/класса.
3. Вычисление значений функции оформить в виде отдельной(ого) функции/класса.
4. Для оценки погрешности и завершения вычислительного процесса использовать правило Рунге.
5. Предусмотреть вывод результатов: значение интеграла, число разбиения интервала интегрирования для достижения требуемой точности.

Вычислительная реализация задачи:

1. Вычислить интеграл, приведенный в таблице 1, точно.
2. Вычислить интеграл по формуле Ньютона – Котеса при $n = 6$.
3. Вычислить интеграл по формулам средних прямоугольников, трапеций и Симпсона при $n = 10$
4. Сравнить результаты с точным значением интеграла.
5. Определить относительную погрешность вычислений для каждого метода.
6. В отчете *отразить последовательные вычисления*.

Необязательное задание

1. Установить сходимость рассматриваемых несобственных интегралов 2 рода (2–3 функции). Если интеграл - расходящийся, выводить сообщение: «Интеграл не существует».

2. Если интеграл сходящийся, реализовать в программе вычисление несобственных интегралов 2 рода (заданными численными методами).
3. Рассмотреть случаи, когда подынтегральная функция терпит

Основные этапы выполнения

Цель работы: найти приближенное значение определенного интеграла с требуемой точностью различными численными методами.

Используемые формулы:

- Табличный интеграл:

$$\int dx = x + C$$

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C$$

- Формула Ньютона – Лейбница (определенный интеграл)

$$\int_a^b f(x) dx = F(x)|_a^b = F(b) - F(a)$$

- Формула шага, где n – количество шагов:

$$h = \frac{b-a}{n}$$

- Формула Ньютона – Котеса при n = 6:

$$\begin{aligned} \int_2^4 f(x) dx &\approx \sum_{i=0}^n f(x_i) c_n^i = c_6^0 f(a) + c_6^1 f(a+h) + c_6^2 f(a+2h) + c_6^3 f(a+3h) + c_6^4 f(a+4h) \\ &+ c_6^5 f(a+5h) + c_6^6 f(b) \\ &= \frac{41(b-a)}{840} f(a) + \frac{216(b-a)}{840} f(a+h) + \frac{27(b-a)}{840} f(a+2h) \\ &+ \frac{272(b-a)}{840} f(a+3h) + \frac{27(b-a)}{840} f(a+4h) + \frac{216(b-a)}{840} f(a+5h) + \frac{216}{840} f(b) \end{aligned}$$

- Формула средних прямоугольников:

$$\begin{aligned} I_{\text{ср.прям.}} &= h \cdot \sum_{i=1}^n y_{i-\frac{1}{2}} = h \cdot (f(a + \frac{h}{2}) + f(a + \frac{3h}{2}) + \dots \\ &+ f(a + \frac{2n-1}{2}h)) \end{aligned}$$

- Формула трапеций:

$$I_{\text{трап.}} = h \cdot (\frac{y_0 + y_n}{2} + \sum_{i=1}^n (a + ih))$$

- Формула Симпсона:

$$I_{\text{Симпсона}} = \frac{h}{3} (y_0 + 4 \sum_{i=1}^{n-1} y_{\text{нечет}} + 2 \sum_{i=2}^{n-2} y_{\text{чет}} + y_n)$$

- Формула сравнения:

$$R = |I_{\text{точн}} - I_{\text{метод}}|$$

- Относительная погрешность

$$\Delta = \frac{|I_{\text{Точн}} - I_{\text{метод}}|}{I_{\text{Точн}}} \cdot 100\%$$

- Правило Рунге:

$$R = \frac{I_{2n} - I_n}{2^k - 1}$$

1. Вычислительная реализация задачи

$$\int_2^4 (-2x^3 - 3x^2 + x + 5)dx$$

1. Точное вычисление:

$$\int_2^4 (-2x^3 - 3x^2 + x + 5)dx$$

$$F(x) = -\frac{x^4}{2} - x^3 + \frac{x^2}{2} + 5x$$

$$F(2) = -4; F(4) = -164$$

$$I_{\text{Точн}} = F(x) = F(4) - F(2) = -164 - (-4) = -160$$

2. Вычисление по формуле Ньютона – Котеса при n = 6:

$$h = \frac{4-2}{6} = \frac{2}{6} = \frac{1}{3}$$

$$\begin{aligned} \int_2^4 (-2x^3 - 3x^2 + x + 5)dx \approx & c_6^0 f(2) + c_6^1 f\left(\frac{7}{3}\right) + c_6^2 f\left(\frac{8}{3}\right) + c_6^3 f(3) + c_6^4 f\left(\frac{10}{3}\right) \\ & + c_6^5 f\left(\frac{11}{3}\right) + c_6^6 f(4) \end{aligned}$$

$$\begin{aligned} I_{\text{cotes}} = 2 \cdot \left(\frac{41}{840} f(2) + \frac{216}{840} f\left(\frac{7}{3}\right) + \frac{27}{840} f\left(\frac{8}{3}\right) + \frac{272}{840} f(3) + \frac{27}{840} f\left(\frac{10}{3}\right) + \frac{216}{840} f\left(\frac{11}{3}\right) \right. \\ \left. + \frac{216}{840} f(4) \right) = -160 \end{aligned}$$

3. Вычисление по формулам

3.1 Средних прямоугольников при n = 10

$$h = \frac{b-a}{n} = \frac{4-2}{10} = \frac{2}{10} = \frac{1}{5}$$

$$\begin{aligned} I_{\text{ср.прям.}} = \frac{1}{5} \cdot \left(f\left(a + \frac{1}{10}\right) + f\left(a + \frac{3}{10}\right) + f\left(a + \frac{5}{10}\right) + f\left(a + \frac{7}{10}\right) + f\left(a + \frac{9}{10}\right) \right. \\ \left. + f\left(a + \frac{11}{10}\right) + f\left(a + \frac{13}{10}\right) + f\left(a + \frac{15}{10}\right) + f\left(a + \frac{17}{10}\right) + f\left(a + \frac{19}{10}\right) \right) \end{aligned}$$

$$\begin{aligned}
I_{\text{ср.прям.}} &= 0.2 \cdot (f(2 + 0.1) + f(2 + 0.3) + f(2 + 0.5) + f(2 + 0.7) + f(2 + 0.9) \\
&\quad + f(2 + 1.1) + f(2 + 1.3) + f(2 + 1.5) + f(2 + 1.7) + f(2 + 1.9)) \\
&= -159.86
\end{aligned}$$

3.2 Трапеций при $n = 10$

$$h = \frac{b - a}{n} = \frac{4 - 2}{10} = \frac{2}{10} = \frac{1}{5}$$

$$\begin{aligned}
I_{\text{трап}} &= 0.2 \left(\frac{f(2) + f(4)}{2} + f(2.2) + f(2.4) + f(2.6) + f(2.8) + f(3) + f(3.2) + f(3.4) \right. \\
&\quad \left. + f(3.6) + f(3.8) \right) = -160.28
\end{aligned}$$

3.3 Симпсона при $n = 10$

$$h = \frac{b - a}{n} = \frac{4 - 2}{10} = \frac{2}{10} = \frac{1}{5}$$

$$\begin{aligned}
I_{\text{Симпсона}} &= \frac{0.2}{3} \cdot (f(2) + 4 \cdot (f(2.2) + f(2.6) + f(3) + f(2.4) + f(2.8)) + 2 \\
&\quad \cdot (f(2.4) + f(2.8) + f(3.2) + f(3.6) + f(4))) = -160
\end{aligned}$$

4. Сравнение результатов точным значением интеграла

▪ Точное значение интеграла равно -160

- Метод Ньютона-Котеса при $n = 6$; $I_{\text{cotes}} = -160$

$$R = |I_{\text{точн}} - I_{\text{cotes}}| = |-160 - (-160)| = 0$$

- Метод средних прямоугольников при $n = 10$; $I_{\text{ср.прям.}} = -159.86$

$$R = |I_{\text{точн}} - I_{\text{ср.прям.}}| = |-160 - (-159.86)| = 0.14$$

- Метод трапеций при $n = 10$; $I_{\text{трап.}} = -160.28$

$$R = |I_{\text{точн}} - I_{\text{трап.}}| = |-160 - (-160.28)| = 0.28$$

- Метод Симпсона при $n = 10$; $I_{\text{Симпсона}} = -160$

$$R = |I_{\text{точн}} - I_{\text{Симпсона}}| = |-160 - (-160)| = 0$$

5. Относительная погрешность каждого метода

- Метод Ньютона-Котеса:

$$R = 0 \Rightarrow \text{погрешности нет}$$

- Метод средних прямоугольников:

$$\Delta = \frac{|160 - 159.86|}{160} \cdot 100\% = 0.0875\%$$

- Метод трапеций:

$$\Delta = \frac{|160 - 160.28|}{160} \cdot 100\% = 0.175\%$$

- Метод Симпсона:

$R = 0 \Rightarrow$ погрешности нет

2. Программная реализация

```
3. import math

def f1(x):
    return x ** 2

def f2(x):
    return math.sin(x)

def f3(x):
    return math.exp(x)

def f4(x):
    return 1 / x ** 2

def f5(x):
    return math.log(x)

functions = [f1, f2, f3, f4, f5]

def rectangle(func, a, b, n, mode="middle"):
    h = (b - a) / n
    result = 0
    if mode == "left":
        for i in range(n):
            result += func(a + i * h)
    elif mode == "right":
        for i in range(1, n + 1):
            result += func(a + i * h)
    else:
        for i in range(n):
            result += func(a + (i + 0.5) * h)
    result *= h
    return result

def trapezoid(func, a, b, n):
    h = (b - a) / n
    result = (func(a) + func(b)) / 2
    for i in range(1, n):
        result += func(a + i * h)
    result *= h
    return result

def simpson(func, a, b, n):
    h = (b - a) / n
    result = func(a) + func(b)
    for i in range(1, n):
        coef = 3 + (-1) ** (i + 1)
        result += coef * func(a + i * h)
    result *= h / 3
    return result

methods = {
    "Метод левых прямоугольников": lambda f, a, b, n: rectangle(f, a,
b, n, mode="left"),
    "Метод правых прямоугольников": lambda f, a, b, n: rectangle(f, a,
b, n, mode="right"),
    "Метод средних прямоугольников": rectangle,
    "Метод трапеций": trapezoid,
    "Метод Симпсона": simpson
}
```



```

def compute_integral(func, a, b, epsilon, method):
    n = 4
    runge_coef = {
        "Метод левых прямоугольников": 1,
        "Метод правых прямоугольников": 1,
        "Метод средних прямоугольников": 3,
        "Метод трапеций": 3,
        "Метод Симпсона": 15
    }
    coef = runge_coef[method]
    result = methods[method](func, a, b, n)
    r = math.inf
    while r > epsilon:
        n *= 2
        new_result = methods[method](func, a, b, n)
        r = abs(new_result - result) / coef
        result = new_result
    return result, n

def check_discontinuity_point(func, x):
    try:
        func(x)
        return False
    except (ZeroDivisionError, ValueError, OverflowError):
        return True

def check_convergence(func, a, b):
    if func == f4:
        if a == 0 or b == 0 or (a < 0 < b):
            return False
        return True
    elif func == f5:
        if a <= 0:
            return False
        return True
    else:
        return True

def check_discontinuity(func, a, b):
    if check_discontinuity_point(func, a) or
    check_discontinuity_point(func, b):
        return True
    if func == f4 and a < 0 < b:
        return True
    return False

def compute_improper_integral(func, a, b, epsilon, method):
    delta = 1e-8
    if check_discontinuity_point(func, a):
        a += delta
    if check_discontinuity_point(func, b):
        b -= delta
    return compute_integral(func, a, b, epsilon, method)

def get_int_in_range(prompt, min_val, max_val):
    while True:
        try:
            val = int(input(prompt))
            if min_val <= val <= max_val:
                return val
            print(f"Ошибка: введите целое число от {min_val} до {max_val}")
        except ValueError:

```

```

        print("Ошибка: пожалуйста, введите целое число")

def get_positive_float(prompt):
    while True:
        try:
            val = float(input(prompt))
            if val > 0:
                return val
            print("Ошибка: точность должна быть строго больше нуля")
        except ValueError:
            print("Ошибка: пожалуйста, введите корректное число")

def get_integration_limits():
    while True:
        try:
            a = float(input("Начальный предел (a): "))
            b = float(input("Конечный предел (b): "))
            if a < b:
                return a, b
            print("Ошибка: начальный предел 'a' должен быть строго меньше конечного предела 'b'. Попробуйте снова")
        except ValueError:
            print("Ошибка: пожалуйста, введите корректные числа")

def main():
    print("\nВыберите функцию:")
    print("1. x^2")
    print("2. sin(x)")
    print("3. e^x")
    print("4. 1/x^2")
    print("5. log(x)")

    func_choice = get_int_in_range("Ваш выбор (1-5): ", 1, 5)
    func = functions[func_choice - 1]

    print("\nВведите пределы интегрирования:")
    a, b = get_integration_limits()

    print("\nВыберите метод интегрирования:")
    for i, method_name in enumerate(methods, 1):
        print(f"{i}. {method_name}")

    method_choice = get_int_in_range("Ваш выбор (1-5): ", 1, 5)
    method = list(methods.keys())[method_choice - 1]

    epsilon = get_positive_float("\nВведите требуемую точность вычислений (>0): ")

    if not check_convergence(func, a, b) or check_discontinuity(func, a, b):
        print("\nИнтеграл не существует (расходится или разрыв на границе)")
    else:
        result, n = compute_improper_integral(func, a, b, epsilon, method)
        print("\nРезультат:")
        print(f"Значение интеграла: {result}")
        print(f"Число разбиений для достижения точности: {n}")

if __name__ == "__main__":
    main()

```

Выберите функцию:

1. x^2
2. $\sin(x)$
3. e^x
4. $1/x^2$
5. $\log(x)$

Ваш выбор (1-5): 1

Введите пределы интегрирования:

Начальный предел (a): 0

Конечный предел (b): 1

Выберите метод интегрирования:

1. Метод левых прямоугольников
2. Метод правых прямоугольников
3. Метод средних прямоугольников
4. Метод трапеций
5. Метод Симпсона

Ваш выбор (1-5): 1

Введите требуемую точность вычислений (>0): 0.01

Результат:

Значение интеграла: 0.3255615234375

Число разбиений для достижения точности: 64

Вывод

Выполнение лабораторной работы по вычислительной математике позволило мне познакомиться с численным интегрированием. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я написала программу для приближенного вычисления интегралов на ЯП Python, используя различные методы (метод прямоугольников, метод трапеций, метод Симпсона). Приобретённый опыт поможет мне при дальнейшем изучении предмета вычислительная математика.